Software Requirements Specification (SRS)

Smart Recruitment & Resume Screening System (SRRSS)

Version 1.0

1. Introduction

1.1. Purpose

This document outlines the specifications for the Smart Recruitment & Resume Screening System, an AI-driven platform designed to automate the hiring process. It enables candidates to apply seamlessly and helps recruiters screen, rank, and schedule interviews with applicants using Natural Language Processing (NLP) to reduce manual effort and bias.

1.2. Scope

- **Automated Resume Parsing:** Extracting details from PDF/DOCX resumes.
- **Semantic Matching:** Ranking candidates based on job description relevance.
- **Workflow Automation:** Scheduling interviews and sending status emails.
- **Admin & Recruiter Panels:** Dashboards for managing jobs and analytics.
- **Secure Data Handling:** Encryption of personal candidate data.

1.3. Intended Audience

Stakeholders (HR Managers), developers, QA testers, UI/UX designers, and college faculty/evaluators.

1.4. Product Features

- **Candidate Portal:** One-click application and profile management.
- **Resume Parser:** AI engine to extract Skills, Education, and Experience.
- **Smart Ranking:** assigning a "Match Score" (0-100%) to every applicant.
- **Bias Detection:** Flagging discriminatory language in job descriptions.
- **Automated Scheduler:** Integration with calendars for interview booking.
- **Notification System:** Real-time email/SMS updates on application status.

- **Analytics Dashboard:** Visualizing "Time-to-Hire" and "Source Quality".
- **Role-Based Access Control:** Distinct views for Admin, Recruiter, and Candidate.

1.5. Non-Functional Requirements

- **Performance:** Resume parsing and scoring must complete in under 3 seconds.
- **Security:** JWT authentication, Data Encryption at Rest, GDPR compliance.
- **Reliability:** 99.9% uptime for the application server.
- **Scalability:** Microservices architecture to handle high volumes of resume uploads.
- **Usability:** Mobile-responsive interface for candidates.

1.6. Architecture Overview

- **Frontend:** React.js (Web Client).
- **Backend:** Node.js / Express (API Gateway).
- **AI Engine:** Python (Flask/FastAPI) with SpaCy/NLTK.
- **Database:** MongoDB (Candidate Data), AWS S3 (File Storage).
- **Hosting:** AWS / Vercel.

1.7. Future Enhancements

- **Video Interview Analysis:** AI analysis of candidate confidence and sentiment.
- **Code Assessment Module:** Built-in compiler for technical testing.
- **Blockchain Verification:** Verifying degree certificates on-chain.

1.8. User Stories for Smart Recruitment System

ID	User Role	Wants To	So That
US001	Candidate	register using email or LinkedIn	I can create a profile quickly.
US002	Candidate	upload my resume in PDF/DOCX	the system can auto-fill my profile.
US003	Recruiter	post a new job description	candidates can start applying.
US004	Recruiter	see a ranked list of candidates	I don't have to read every resume manually.
US005	Candidate	track my application status	I know if I have been shortlisted or rejected.
US006	Recruiter	filter candidates by specific skills	I can find specialists (e.g., "React Developers") easily.
US007	Admin	view system usage statistics	I can monitor how many jobs are being closed.
US008	Recruiter	schedule an interview slot	the candidate receives an invite automatically.
US009	Candidate	receive email notifications	I don't miss any interview requests.
US010	Admin	manage user roles and permissions	unauthorized users cannot access sensitive data.

US011	Recruiter	download candidate reports	I can share them with hiring managers.
US012	System	flag biased job descriptions	the hiring process remains fair and inclusive.

1.9. Smart Recruitment System Project Plan

Overview

- **Project Duration:** Approximately 8-10 weeks.
- **Development Methodology:** Agile (Scrum-based with 1-week sprints).

Phase 1: Requirement Analysis & Planning (Week 1)

- **Deliverables:** Finalized SRS, Tech Stack Selection, Project Roadmap.
- **Tasks:** Stakeholder meetings, Define MVP scope, Setup GitHub/Jira.

Phase 2: Design (Week 2)

- **Deliverables:** UI Wireframes (Figma), Database Schema (MongoDB), API Documentation.
- **Tasks:** Designing the "Recruiter Dashboard" and "Resume Parsing Pipeline".

Phase 3: Backend & AI Development (Week 3-4)

- **Deliverables:** REST APIs (Node.js), Resume Parser Microservice (Python), Auth System.
- **Tasks:**
  - Setup Express server and MongoDB connection.
  - Implement NLP logic using SpaCy/TF-IDF.
  - Unit testing for parser accuracy.

Phase 4: Frontend Development (Week 5-6)

- **Deliverables:** React Web App, Candidate Portal, Recruiter Dashboard.
- **Tasks:**
  - Integrate frontend with Node.js APIs.
  - Build "Drag & Drop" resume upload interface.
  - Visualize candidate ranking scores.

Phase 5: Integration & Testing (Week 7)

- **Deliverables:** Integrated System, Test Cases Document, Bug Reports.
- **Tasks:**
  - End-to-end testing (Upload -> Parse -> Rank).
  - Cross-browser compatibility testing.

Phase 6: DevOps & Deployment (Week 8)

- **Deliverables:** Live Deployment (Vercel/Heroku), Docker Containers.
- **Tasks:**
  - Configure CI/CD pipelines.
  - Setup MongoDB Atlas (Cloud DB).

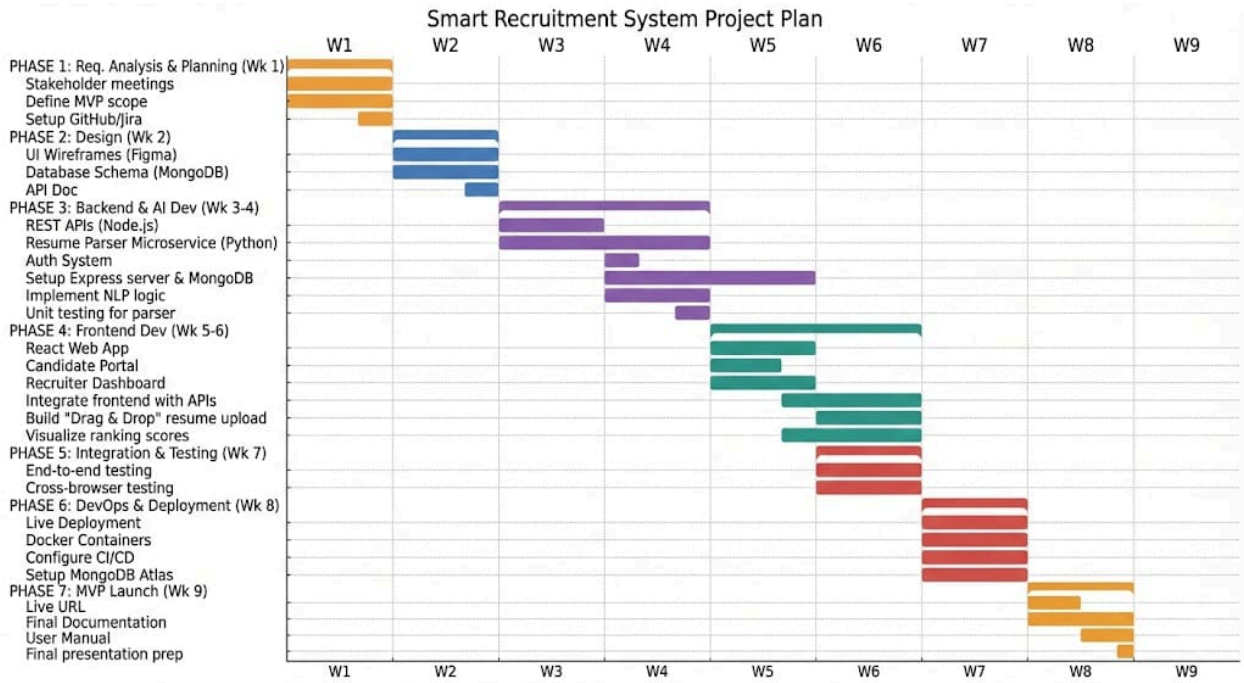
Phase 7: MVP Launch (Week 9)

- **Deliverables:** Live URL, Final Documentation, User Manual.
- **Tasks:** Final presentation preparation.

Tools & Technologies

Area	Tools / Stack
Frontend	React.js, Tailwind CSS, Redux
Backend	Node.js, Express.js
AI/ML Engine	Python, Flask, SpaCy, Scikit-learn
Database	MongoDB (NoSQL), AWS S3 (Files)
DevOps	Docker, GitHub Actions, Vercel/Heroku
Design	Figma, Canva
Project Mgmt	Jira / Trello
Testing	Jest, Postman

Timeline Overview

Phase	Duration (Weeks)	Sprint(s)
Req. Analysis	1	Sprint 1
Design	1	Sprint 2
Backend & AI	2	Sprint 3-4
Frontend Dev	2	Sprint 5-6
Testing	1	Sprint 7
Deployment	1	Sprint 8

MVP Launch	1	Sprint 9